IUT Paul SABATIER

Dépt Informatique

TP 1 : Initiation DevOps — GitLab CI/CD & Déploiement

Objectif du TP

Ce TP vous guide à travers les étapes du cycle DevOps :

1. Développer du code avec des tests unitaires.
2. Gérer les versions du code avec Git et GitLab.
3. Conteneuriser votre application avec Docker.
4. Automatiser les tests et le build via GitLab CI/CD.
5. Gérer des releases versionnées.
6. Déployer automatiquement votre application sur un serveur avec Ansible.

Prérequis et installations

- IDE : Visual Studio Code ou PyCharm
- Docker et Docker-compose
- Git et Python
- Un compte GitLab :
  - IUT : https://gitlab.info.iut-tlse3.fr/
  - Ou GitLab standard : https://gitlab.com/users/sign_in/

Etape 1 : Créer le projet

- Dans votre IDE, créez la structure de code suivante :
  - *src/calculs/basic.py*
  - *tests/test_basic.py*
  - *app.py*
  - *requirements.txt*
  - *.gitignore*
  - *tests/__init__.py*
  - *src/__init__.py*
  - *src/calculs/__init__.py*

- Implémenter deux fonctions simples dans basic.py : add (addition de deux nombres) et mul (multiplication de deux nombres).
- Dans app.py, implémenter le programme principal qui appelle les deux fonctions.
- Créer et activer un environnement virtuel avec votre IDE, et lancer le programme app.py pour vérifier le bon déroulement du programme.

Etape 2 : Implémenter des tests

- Dans requirements.txt, ajouter la librairie "pytest" qui va permettre d'implémenter des tests.
- Installer la dépendance :
  - pip install –r requiments.txt
- Implémenter plusieurs cas de tests pour les fonctions add et mul dans le fichier test_basic.py. Exemple :

```
def test_add_positive_numbers():
        assert add(2, 3) == 5
...
```

- Lancer les tests : "pytest –v". Et vérifier que tous les tests retournent PASSED.
- Introduire volontairement une erreur de calcul dans la fonction add (par exemple remplacer l'opérateur + par – et relancer les tests). Plusieurs tests sont dans l'état FAILED.
- Remettre le code à la version correcte.

Étape 3 : Packager l'application avec Docker

- Créer un fichier Dockerfile et un fichier docker-compose.yml pour l'application.
- Lancer "docker-compose up –build" et vérifier le bon déroulement de l'exécution.

Étape 4 : Gestion des versions avec Git et Gitlab

- Initialiser Git :
  - git init
- Dans .gitignore, ajouter les fichiers qui seront ignorés par git (__pycache__, .venv, …).
- Faire un premier git commit :
  - git add .
  - git commit –m "initial commit"
- Créer un projet vide dans Gitlab :
  - Visibility Level : Private:
  - Décocher : Initialize repository with a README
- Exécuter les commandes proposées par GitLab pour pousser votre code, notamment :

- Git local setup.
- Onglet HTTPS :
  - Push an existing Git repository

Étape 5 : Implémenter une nouvelle fonctionnalité

- Créer une branche feature/power :
  - git checkout -b feature/power
- Ajouter une fonction pow à votre code (calcul de puissance).
- Écrire des tests pour cette fonction.
- Pousser la branche vers Gitlab.
  - git add .
  - git commit -m "implement power"
  - git push --set-upstream origin feature/power
- Créer une Merge Request dans Gitlab et la fusionner dans la branche main.
- En local revenez sur la branche main :
  - git checkout main
- Se synchroniser avec gitlab via :
  - git pull

Étape 6 : Conflit de merge

- Créer deux branches à partir du même commit :
  - feature/rename-mul : renommer la fonction mul en multiply.
  - feature/change-mul : changer la fonction mul pour (a*b)+1 et renommer la fonction mul_plus_one.
- Fusionner rename-mul dans main via une Merge Request.
- Tenter de fusionner change-mul via une Merge Request→ GitLab signale un conflit.
- Résoudre le conflit

Étape 7 : Rebase

- Créer une branche feature/rebase-demo dans lequelle vous ajoutez un fichier Readme.md au projet et faire un commit (pas un push).
- Revenir à la branche main et ajouter quelques commentaires dans votre code et faire un commit et un push.

- Faire un rebase de la branche feature/rebase-demo sur origin/main pour rattraper le retard.
    - git checkout feature/rebase-demo
    - git rebase origin/main
    - git push --set-upstream origin feature/demo-rebase
    - git checkout main
- Fusionner la branche feature/rebase-demo sur main via merge request.

Étape 8 — GitLab CI/CD : tests

- Créer un fichier .gitlab-ci.yml dans la racine de votre projet.
- Ajouter un job test qui installe pytest, exécute les tests et produit reports/junit.xml :

```yaml
# Définition des étapes du pipeline, ici une seule étape : "test"
stages: [test]
# Définition d'un job appelé "test"
test:
  # Ce job appartient à l'étape "test"
  stage: test
  # Utilise une image Docker officielle Python 3.13 en version légère (slim)
  image: python:3.13-slim
  # Règles qui déterminent si le job doit être exécuté
  rules:
    # Le job s'exécute si le pipeline vient d'une merge request
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'
    # Le job s'exécute si un commit est poussé sur une branche
    - if: '$CI_COMMIT_BRANCH'
  # Commandes exécutées avant le script principal
  before_script:
    # Installe les dépendances Python listées dans requirements.txt
    - pip install --no-cache-dir -r requirements.txt
    # Crée le dossier "reports" si besoin
    - mkdir -p reports
  # Commandes principales du job
  script:
    # Lance les tests avec pytest et génère un rapport JUnit XML
    - pytest --junitxml=reports/junit.xml
  # Définition des fichiers sauvegardés après l'exécution du job
  artifacts:
    # Les artifacts sont sauvegardés même si le job échoue
    when: always
    # Section dédiée aux rapports de tests
    reports:
      # Sauvegarde le rapport JUnit généré par pytest
      junit: reports/junit.xml
  # Si le job échoue, il fait échouer tout le pipeline (échec non toléré)
  allow_failure: false
```

- Vérifier que GitLab affiche le rapport de tests.
- Vérifier que la branche main est protégée pour éviter le merge de code dont les tests ne passent pas :
    - Projet → **Settings → Repository → Protected branches,** Branche : main, **Allowed to push** : Maintainers, **Allowed to merge** : *Maintainers*
    - Projet → **Settings → General → Merge requests → Merge checks,** Cocher **Pipelines must succeed**
- Essayer de faire une MR avec du code dont les tests sont FAIL => le merge est bloqué
- Corriger l'erreur (tests PASS) puis faire le Merge.

Etape 9 : build and push

- Nous allons dans cette partie ajouter un job qui automatise le build de l'image de docker et son push dans un Repo d'images docker : Dockerhub.
- Dans Gitlab -> Settings -> CI/CD -> Variables, ajouter les variables suivantes dont les valeurs seront fournies par l'enseignant. Ces variables vont contenir les informations pour s'authentifier dans Dockerhub depuis le job. Vous pouvez également utiliser vos propres informations si vous utilisez votre propre compte Dockerhub. Pendant l'ajout des variables décocher le champ "protect variable".
    - DOCKERHUB_REPO
    - DOCKERHUB_TOKEN
    - DOCKERHUB_USERNAME
- Ajouter au fichier .gitlab-ci.yaml la section suivante pour ajouter le job build docker :

```yaml
# Définition des étapes du pipeline, ici deux étapes : "test" et "build"
stages: [test,build]
# Définition d'un job appelé "test"
test: …

# Définition d'un job appelé "build"
build:
  stage: build # Ce job appartient au stage "build"
  image: docker:26.1.4 # Image Docker utilisée pour exécuter le job
  # Service Docker-in-Docker pour construire des images
  services:
    - name: docker:26.1.4-dind # Service Docker utilisé pour le build
      alias: docker # Alias "docker" pour ce service
  rules: # Règles de déclenchement du job
    # Exécuter ce job si on est sur la branche "main"
    - if: '$CI_COMMIT_BRANCH == "main"'
    # Exécuter aussi si le pipeline est déclenché par un tag
    - if: '$CI_COMMIT_TAG'
  # Variables d'environnement spécifiques au job
  variables:
    DOCKER_HOST: tcp://docker:2375 # Adresse du daemon Docker
    DOCKER_DRIVER: overlay2
    DOCKER_TLS_CERTDIR: ""
    # Chemin vers le Dockerfile
    DOCKERFILE_PATH: "Dockerfile"
    # Code étudiant
    STUDENT_CODE: "MKAN" # <---------------------- À remplacer par votre code étudiant
    # Préfixe du nom des images (inclut le code étudiant)
    IMAGE_PREFIX: "$DOCKERHUB_USERNAME/$DOCKERHUB_REPO:$STUDENT_CODE"
  # Étapes à exécuter avant le script principal
  before_script:
    # Authentification à DockerHub avec utilisateur/token
    - echo "$DOCKERHUB_TOKEN" | docker login -u "$DOCKERHUB_USERNAME" --password-stdin
  # Script principal du job
  script:
    # Construction de l'image Docker avec le SHA du commit comme tag
    - docker build --pull -f "$DOCKERFILE_PATH" -t "$IMAGE_PREFIX-$CI_COMMIT_SHORT_SHA" .
    # Envoi de l'image construite sur DockerHub
    - docker push "$IMAGE_PREFIX-$CI_COMMIT_SHORT_SHA"
    # Bloc conditionnel : si le pipeline est déclenché par un tag
    - |
      if [[ -n "$CI_COMMIT_TAG" ]]; then
        # Affiche quel tag a déclenché le build
        echo "Build triggered by tag: $CI_COMMIT_TAG"
        # Retag l'image avec le nom du tag Git
        docker tag "$IMAGE_PREFIX-$CI_COMMIT_SHORT_SHA" "$IMAGE_PREFIX-$CI_COMMIT_TAG"
        # Push de l'image taguée avec le tag Git sur DockerHub
        docker push "$IMAGE_PREFIX-$CI_COMMIT_TAG"
      fi
```

- Pousser vers Gitlab et vérifier le succès de l'opération et de présence de l'image Docker dans Dockerhub

Étape 10 : Gestion des releases

- Créez un tag v1.0.0 dans votre projet GitLab.
- Vérifier qu'un job de build s'est déclenché et qu'une image a été ajoutée à Dockerhub avec le tag v1.0.0

Etape 11 : Déploiement

- Nous allons déployer l'application dans une instance distante dans le cloud en utilisant Ansible.
- Ajouter dans votre projet la structure suivante :

- ansible/deploy.yml
- ansible/inventory
- ansible/docker-compose.yml

- Dans Gitlab -> Settings -> CI/CD -> Variables, ajouter la variable suivante dont la valeur sera fournie par l'enseignant. Cette variable contient la clé ssh pour se connecter à l'instance et effectuer le déploiement. Pendant l'ajout de la variable décocher le champ "protect variable".
  - EC2_SSH_KEY
- Editer le fichier ansible/docker-compose.yml.

```yaml
services:
  ▷ Run Service
  app-MKAN:   # <------ Remplacer MKAN par votre code étudiant
    image: "{{ image }}"
    command: ["python", "app.py"]
```

- Editer le fichier inventory. Ce fichier contient les informations pour se connecter à l'instance en SSH.

```
[appserver]
ec2 ansible_host=15.188.76.73 ansible_user=ubuntu ansible_ssh_private_key_file=~/.ssh/id_rsa
```

- Editer le fichier deploy.yml. Ce fichier est un playbook Ansible qui contient les instructions pour effectuer le déploiement sur l'instance :

```yaml
---
- name: Deploy with Docker Compose
  hosts: appserver
  become: true

  tasks:
    - name: Ensure app directory exists
      ansible.builtin.file:
        path: /opt/{{ student_code }}/app
        state: directory
        mode: '0755'

    - name: Copy docker-compose.yml to server
      ansible.builtin.template:
        src: docker-compose.yml
        dest: /opt/{{ student_code }}/app/docker-compose.yml
        mode: "0644"

    - name: Docker login to Docker Hub
      community.docker.docker_login:
        registry_url: "https://index.docker.io/v1/"
        username: "{{ dockerhub_username }}"
        password: "{{ dockerhub_token }}"

    - name: Run docker compose up
      ansible.builtin.command:
        cmd: docker-compose -f /opt/{{ student_code }}/app/docker-compose.yml up -d
        chdir: /opt/{{ student_code }}/app
```

- Mettre à jour le fichier .gitlab-ci.yml pour ajouter un job de déploiement :

```yaml
# Définition d'un job appelé "deploy"
deploy:
  # Ce job appartient au stage "deploy"
  stage: deploy
  # Utilise une image Docker officielle Python 3.12
  image: python:3.12
  variables:
    EC2_PUBLIC_IP: 15.188.76.73 # <---------------------- À remplacer par l'IP fournie par l'enseignant
    STUDENT_CODE: "MKAN" # <---------------------- À remplacer par votre code étudiant
    IMAGE_PREFIX: "$DOCKERHUB_USERNAME/$DOCKERHUB_REPO:$STUDENT_CODE"
  # Règles qui déterminent si le job doit être exécuté
  rules:
    - if: '$CI_COMMIT_TAG'
  # Étapes à exécuter avant le script principal
  before_script:
    # Installe Ansible
  - pip install --no-cache-dir ansible
  - mkdir -p ~/.ssh
    # Configure la clé SSH pour se connecter au serveur EC2
  - echo "$EC2_SSH_KEY" > ~/.ssh/id_rsa
    # Donne les permissions appropriées à la clé privée
  - chmod 600 ~/.ssh/id_rsa
    # Ajoute l'IP du serveur EC2 aux hôtes connus pour éviter les prompts interactifs
  - ssh-keyscan -H $EC2_PUBLIC_IP >> ~/.ssh/known_hosts
  script:
    # Commande pour exécuter le playbook Ansible de déploiement
    - ansible-playbook -i ansible/inventory ansible/deploy.yml --extra-vars "image=$IMAGE_PREFIX-$CI_COMMIT_TAG student_code=$STUDENT_CODE
    dockerhub_username=$DOCKERHUB_USERNAME dockerhub_token=$DOCKERHUB_TOKEN"
```

- Pousser les modifications vers GitLab, créer un Tag (exemple v1.0.1) et vérifier la bonne exécution du job.
- Vérifier avec l'enseignant que l'application est déployée sur l'instance.